

Тема 2.4

Словари. Множества

Что такое словарь?

Задача. В файле находится список слов, среди которых есть повторяющиеся. Каждое слово записано в отдельной строке. Построить **алфавитно-частотный словарь**: список слов в алфавитном порядке, справа от каждого слова должно быть указано, сколько раз оно встречается в исходном файле.



Какая структура данных нужна?

```
print ( D [ "бегемот" ] )
```

поиск не по индексу,
а по слову (*ключу*)

ключ → **значение**
(отображение, *map*)

Словарь — это неупорядоченный набор элементов, в котором доступ к элементу выполняется по ключу.

Алгоритм (псевдокод)

ключ → значение

слово

счётчик



Как выбрать ключ и значение?

```
создать пустой словарь
while есть слова в файле:
    прочитать очередное слово
    if слово есть в словаре:
        увеличить на 1 счётчик
        для этого слова
    else:
        добавить слово в словарь
        записать 1 в счётчик слова
```

Работа со словарями в Python

Создание:

```
D = { }          # пустой словарь
```

```
D = { "бегемот" : 0, "пароход" : 2 }
```

Добавление (изменение) элемента:

```
D [ "самолёт" ] = 1
```



Создаётся новый элемент!

```
D [ "самолёт" ] += 1
```

ошибка, если
ключа нет



Нужно проверить, есть ли элемент!

Работа со словарями в Python

Изменение с проверкой:

```
if "самолёт" in D:  
    D["самолёт"] += 1  
else:  
    D["самолёт"] = 1
```

или так:

```
D["самолёт"] = D.get( "самолёт", 0 ) + 1
```

получить значение
по ключу

значение по
умолчанию (если
ключа нет)

Основной цикл

```
D = {}  
F = open ( "input.txt" )  
while True:  
    word = F.readline() .strip()  
    if not word:  
        break  
    D[word] = D.get ( word, 0 ) + 1  
F.close()
```

создать пустой
словарь

прочитать
строку, убрать
"**\n**" в конце

кончились данные – выход

увеличить
счётчик слова

Вывод результата

Получить массив всех ключей:

```
allKeys = D.keys()
```

отсортировать ключи:

```
sortKeys = sorted(D.keys())
```

или так:

```
sortKeys = sorted(D)
```

Вывод результата:

```
F = open ( "output.txt", "w" )  
for k in sorted(D):  
    F.write ( "{}: {}".format(k, D[k]) )  
F.close()
```

```
пароход: 12  
самолёт: 20
```

Перебор значений:

```
for i in D.values():  
    print ( i )
```

Перебор ключей и значений:

```
for k, v in D.items():  
    print ( k, "->", v )
```

список пар
(ключ, значение)

Словарь и массив пар

Массив (список) пар «ключ-значение»:

```
A = list(D.items())
```

список пар
(ключ, значение)

```
D = {"бам": 2, "там": 3}
```



A[0] кортеж A[1]

```
A = [ ("бам", 2), ("там", 3) ]
```

A[0][0] A[0][1]

Сортировка:

```
for i in range(N-1):  
    for j in range(N-2, i-1, -1):  
        if A[j][1] < A[j+1][1]:  
            A[j], A[j+1] = A[j+1], A[j]
```

по значению!

Словарь и массив пар

Сортировка:

```
A.sort()
```

по ключам, если ключи
равны – по значениям

```
A.sort( key = lambda x: x[0])
```

по ключам

```
A.sort( key = lambda x: x[1])
```

по значениям

```
A.sort( key = lambda x: (x[1], x[0]))
```

по значениям, если они
равны – по ключам

```
A.sort( key = lambda x: (-x[1], x[0]))
```

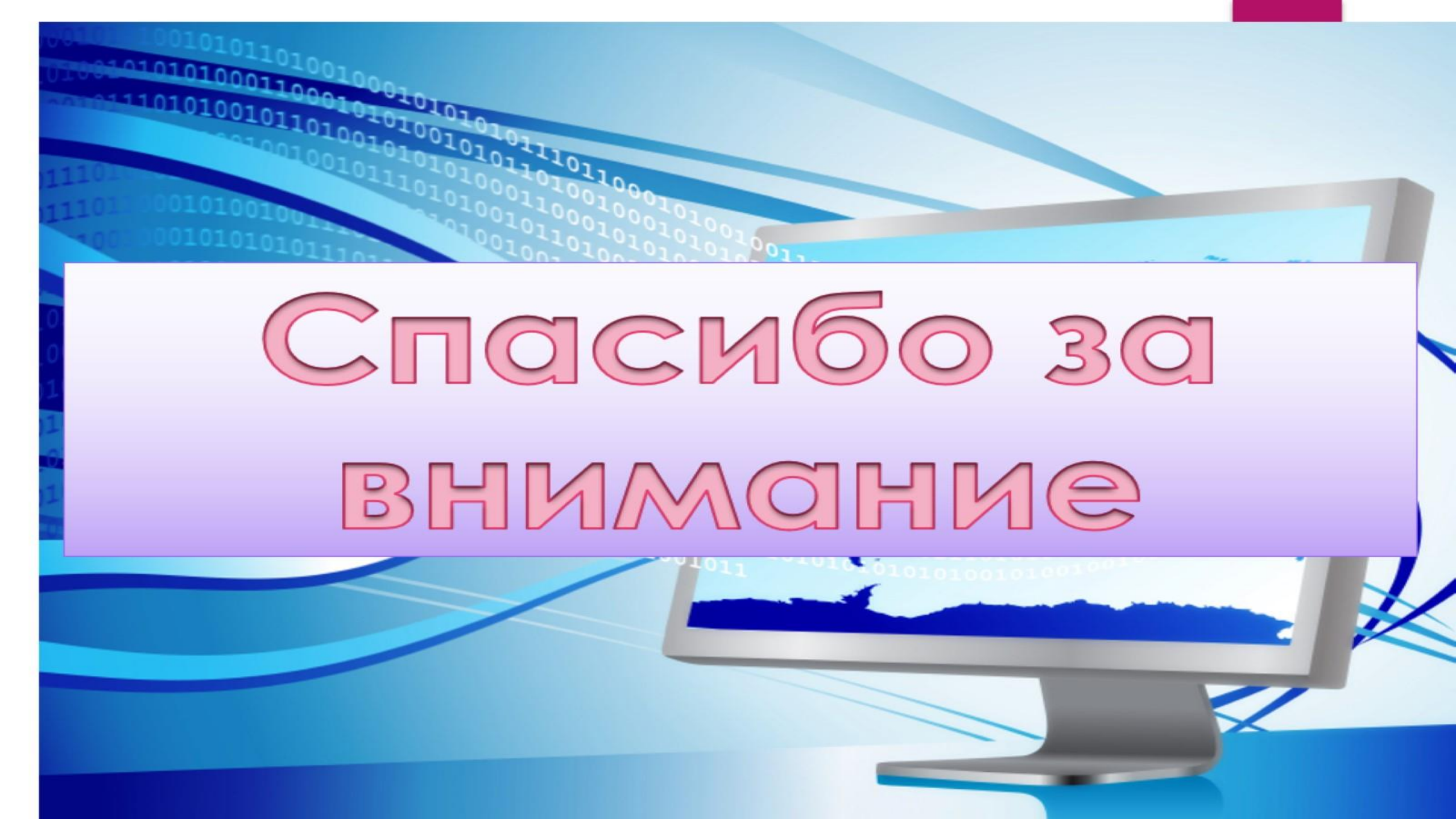
Словарь и массив пар

Вывод массива пар

```
for x in A:  
    print( x[0], ":", x[1], sep="" )
```

или так

```
for x in A:  
    print( "{}: {}".format(x[0], x[1]) )
```

The background features a light blue gradient with several curved, darker blue lines. Scattered throughout are strings of binary code (0s and 1s) in a light blue font. Two computer monitors are visible: one in the upper right and one in the lower right. The lower monitor's screen shows a dark blue silhouette of a landscape with a jagged horizon line.

Спасибо за
внимание